

Case Study: E-commerce Management (Products Collection)

Find Operations

1) Find all products in the collection.

```
> _MONGOSH  
> // Find all products in the collection  
> db.products.find()
```

2) Display all products in a well-formatted structure.

```
> db.products.find().pretty()
```

3) Find a single product using its email/contact (or unique field like product name).

```
> db.products.find({name:"Headphones"})
```

4) Find products where category is "Electronics".

```
> db.products.find({category:"Electronics"})
```

5) Find products where price is less than 5000.

```
> db.products.find({price:{$lt:5000}})
```

6) Find products where price is less than or equal to 5000.

```
> db.products.find({price:{$lte:5000}})
```

7) Find products where price is greater than 5000.

```
> db.products.find({price:{$gt:5000}})
```

8) Find products where price is greater than or equal to 5000.

```
> db.products.find({price:{$gte:5000}})
```

9) Find products where price is not equal to 5000.

```
> db.products.find({price:{$ne:5000}})
```

10) Find products where price is in the list [1000, 2000, 5000].

```
> db.products.find({price:{$in:[1000,2000,5000]}})
```

11) Find products where price is not in the list [1000, 2000, 5000].

```
> db.products.find({price:{$nin:[1000,2000,5000]}})
```

12) Find products where category is "Electronics" AND price is 5000.

```
> db.products.find({$and:[{category:"Electronics"},{price:5000}]})
```

13) Find products where category is "Electronics" OR brand is "Apple".

```
> db.products.find({$or:[{category:"Electronics"},{brand:"Apple"}]})
```

14) Find products where price is greater than 5000 OR brand is "Samsung".

```
> db.products.find({$or:[{price:{$gt:5000}},{brand:"Samsung"}]})
```

15) Find products where price is between 5000 and 20000.

```
> db.products.find({price:{$gt:5000,$lt:20000}})
```

16) Find products where price is NOT greater than 5000.

```
> db.products.find({price:{$not:{$gt:5000}}})
```

Update Operations

1) Update product name using a specific product ID.

```
> db.products.updateOne({_id:ObjectId("6a0fe91358283024458b00c0")},{ $set:{name:"Headphones"}})
```

2) Update stock for products where price is less than 5000.

```
> db.products.updateMany({price:{$lt:5000}},{ $set:{stock:25}})
```

3) Update price for one product that meets a condition.

```
> db.products.updateOne({brand:"IKEA"},{ $set:{price:25000}})
```

4) Update stock for multiple products where category is "Electronics".

```
> db.products.updateMany({category:"Electronics"},{ $set:{stock:30}})
```

5) Replace an entire product document with new details.

```
> db.products.findOneAndReplace({_id:ObjectId("6a0fe91358283024458b00c0")},{name:"Speaker",category:"Electronics",price:10000,stock:20,brand:"Samsung"})
```

Delete Operations

1) Delete a product using its name or email.

```
> db.products.deleteOne({name:"Speaker"})
```

2) Delete one product where price is less than 2000.

```
> db.products.deleteOne({price:{$lt:2000}})
```

3) Delete all products from the collection (like truncate).

```
> db.products.deleteMany({})
```

Projection & Filtering

1) Display only product name and price fields.

```
> db.products.find({}, {name:1, price:1, _id:0})
```

2) Display product name and price without showing ID.

```
> db.products.find({}, {name:1, price:1, _id:0})
```

Limit & Pagination

1) Display only the first 5 products.

```
> db.products.find().limit(5)
```

2) Skip first 5 products and display next 5 products.

```
> db.products.find().limit(5).skip(5)
```

Sorting

1) Sort products by price in ascending order.

```
> db.products.find().sort({price:1})
```

2) Sort products by price in descending order.

```
> db.products.find().sort({price:-1})
```